

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-abf9da31b8d2448f8.jsonl`  
- SHA-256: `620e827e9627b45440cfb0d361db73015d06462b82dfbcc14054def470b9525c`  
- Source modified: `2026-07-02T03:04:04+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_41c8dc8a-8a4`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T03:02:14.982Z)

You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is 2026-07-02.
A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/khelsutra (avidullu/khelsutra):

```
{  
  "title": "Issue #101 fixed at source but open: raw non-JSON 5xx body still rendered verbatim",  
  "category": "critical-fix",  
  "severity": "P2",  
  "file": "web/src/lib/errors.ts",  
  "line": 33,  
  "evidence": "The original 524 root cause is gone: /api/compile is now async job-polled (web/src/api/client.ts:172-221, engine #329). But the client-side defense the issue asks for is absent: classifyError (errors.ts:33-35) passes any 5xx body through parseDetail, which returns a non-JSON body raw (errors.ts:7-15), and errorText.ts:12 renders e.detail verbatim ? a Cloudflare/proxy HTML error page still prints as a wall of markup in the UI on any API call (poll, upload, query).",  
  "proposed_fix": "Quick win: in classifyError, when status >= 500 and the body is not JSON (or starts with '<'), return the friendly networkError/timeout code instead of code 'engine' with the raw body. Then update/close #101 noting async compile removed the 524 at source. The 'probe for the finished reel on timeout' bonus can stay as a follow-up.",  
  "effort": "S",  
  "tracked_in": "khelsutra#101"  
}
```

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator, C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field; non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup', C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).

VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training, product/strategy/licensing, real cloud spend, counsel ToS/DPA?

HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you personally saw the defect/debt in current code; if you cannot reproduce the claim from the code, say REFUTED; if a commit fixed it, ALREADY_FIXED.

2. user (2026-07-02T03:02:22.114Z)

```

1      // Error helpers shared by the ingest flow and the build flow. Kept i18n-FREE: callers
map an
2      // IngestErrorCode to a localized string (so hooks stay testable without a translation
context),
3      // while the engine's own `detail` text passes through verbatim (it's the most specific
message).
4      import { ApiError } from "../api/client";
5
6      /** Pull FastAPI's {"detail": "..."} message out of an error body, else return the raw
body. */
7      export function parseDetail(body: string): string {
8          try {
9              const j = JSON.parse(body) as { detail?: unknown };
10             if (j && typeof j.detail === "string") return j.detail;
11         } catch {
12             /* not JSON ? fall through to the raw body */
13         }
14         return body;
15     }
16
17     /** Stable, localizable failure categories surfaced by the ingest flow.
18     * `engine` carries the engine's own message in `detail`; the rest are localized by
code. */
19     export type IngestErrorCode = "invalidUri" | "networkError" | "emptyResult" | "timeout"
| "engine";
20
21     export interface IngestError {
22         code: IngestErrorCode;
23         /** Present for `engine` ? the verbatim FastAPI detail / status; rendered as-is, not
translated. */
24         detail?: string;
25     }
26
27     /** Classify a thrown client error into a stable code. A network throw (ApiError with no
`status`)
28     * is the common "engine unreachable" case ? its own friendly code; any HTTP error
surfaces the
29     * engine's `detail` (e.g. an unsupported-scheme 400, a not-found 404). */
30     export function classifyError(err: unknown): IngestError {
31         if (err instanceof ApiError) {
32             if (err.status === undefined) return { code: "networkError" }; // fetch threw ?
engine truly unreachable
33             if (err.body) {
34                 const detail = parseDetail(err.body);
35                 if (detail) return { code: "engine", detail }; // a real engine message (e.g. a
400 unsupported-scheme)
36             }
37             // A bare 5xx with no body is almost always the engine being down / a gateway that
can't reach it
38             // (e.g. the dev proxy when :8000 is off) ? for a non-technical user a friendly "is
it running?"
39             // beats a cryptic "HTTP 500". A 4xx with no body keeps the status (it's a client-
side mistake).
40             if (err.status >= 500) return { code: "networkError" };
41             return { code: "engine", detail: `HTTP ${err.status}` };
42         }
43         return { code: "engine", detail: String(err) };
44     }
45

```

3. user (2026-07-02T03:02:23.563Z)

No files found

4. user (2026-07-02T03:02:29.550Z)

Found 15 files

```
khelsutra\web\src\components\IngestPanel.tsx
khelsutra\web\src\components\IngestPanel.test.tsx
khelsutra\web\src\api\client.test.ts
khelsutra\web\src\api\client.ts
khelsutra\web\src\App.tsx
khelsutra\web\src\components\build-error.test.tsx
khelsutra\web\src\hooks\useIngestJob.ts
khelsutra\web\src\lib\errorText.test.ts
khelsutra\web\src\hooks\useUpload.test.ts
khelsutra\web\src\hooks\useUpload.ts
khelsutra\web\src\api\upload.test.ts
khelsutra\web\src\lib\errorText.ts
khelsutra\web\src\hooks\useIngestJob.test.ts
khelsutra\web\src\lib\errors.test.ts
khelsutra\web\src\lib\errors.ts
```

5. user (2026-07-02T03:02:29.972Z)

```
1      // Typed client for the engine FastAPI. Default = SAME-ORIGIN relative `/api/*` (the
single-origin
2      // appliance reverse-proxies UI + engine behind one origin; in dev Vite proxies `/api` ?
:8000).
3      // Every call carries a correlation id (header + logs) so a journey is traceable end to
end.
4      import { log, newCorrelationId } from "../lib/log";
5      import type {
6          RallySegment,
7          QueryResponse,
8          CompileResult,
9          EngineConfig,
10         EngineCapabilities,
11         ProcessResponse,
12         JobStatus,
13         UploadInitResponse,
14         UploadPartResponse,
15         UploadCompleteResponse,
16         VideoSummary,
17         VideosResponse,
18         AnnotationsResponse,
19         AssetResponse,
20     } from "../types";
21
22     /** Resolve the engine API base. Default same-origin `/api`. For a SPLIT deploy ? SPA
on
23     * Cloudflare Pages + the engine on a separate `api.*` cloudflared tunnel
(HOSTING_PLAN.md $2) ?
24     * set `VITE_API_BASE` at build time to the engine's API base **including the `/api`
path**
25     * (e.g. `https://api.khelsutra.guru/api`), and lock the engine's
`RALLY_CORS_ALLOW_ORIGINS` to the
26     * Pages origin + put CF Access in front of both hostnames. A trailing slash is trimmed
so
27     * `` `${base}/process` `` never double-slashes. */
28     export function apiBase(): string {
29         return (import.meta.env.VITE_API_BASE ?? "/api").replace(/\/+$/, "");
30     }
31
32     const BASE = apiBase();
33
34     export class ApiError extends Error {
35         readonly cid?: string;
36         readonly status?: number;
37         readonly body?: string;
38         constructor(message: string, opts: { cid?: string; status?: number; body?: string;
```

```

cause?: unknown } = {}) {
39     super(message);
40     this.name = "ApiError";
41     this.cid = opts.cid;
42     this.status = opts.status;
43     this.body = opts.body;
44     if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
45 }
46 }
47
48 interface RequestOpts {
49     method?: string;
50     body?: string;
51     headers?: Record<string, string>;
52     /** Correlation id to thread through this call; auto-generated if absent. */
53     cid?: string;
54 }
55
56 async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
57     const cid = opts.cid ?? newCorrelationId();
58     const url = `${BASE}${path}`;
59     const method = opts.method ?? "GET";
60     log("info", "api.request", { cid, method, url });
61
62     let res: Response;
63     try {
64         res = await fetch(url, {
65             method,
66             body: opts.body,
67             headers: { "content-type": "application/json", "x-correlation-id": cid,
68                 ...(opts.headers ?? {}) },
69         });
69     } catch (err) {
70         // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
71         log("error", "api.network_error", { cid, method, url, error: String(err) });
72         throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
73     }
74
75     if (!res.ok) {
76         const body = await res.text().catch(() => "");
77         log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
78         throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
79     }
80
81     const data = (await res.json()) as T;
82     log("info", "api.response", { cid, method, url, status: res.status });
83     return data;
84 }
85
86 /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
for
87     * processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-
fails 400
88     * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
ApiError. */
89 export async function processVideo(
90     videoPath: string,
91     cid?: string,
92     locale?: string,
93     compute?: string,
94 ): Promise<ProcessResponse> {
95     // `locale` (the UI language) is recorded for PMF analytics (which markets use the
tool). `compute`

```

```

96      // is the compute-picker choice ("local" | "cloud") ? where DETECT runs; omitted ? the
server
97      // default. Both optional + omitted when absent (JSON.stringify drops undefined).
98      return request<ProcessResponse>("/process", {
99          method: "POST",
100          body: JSON.stringify({ video_path: videoPath, locale, compute }),
101          cid,
102      });
103  }
104
105  /** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
106  export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
107      return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
});
108  }
109
110  /** List indexed videos, newest first ? the "my videos" library. GET /api/videos
[main.py:447].
111      * Reload-survival: a past match can be reopened without re-processing. Returns [] on
an empty box. */
112  export async function listVideos(cid?: string): Promise<VideoSummary[]> {
113      const data = await request<VideosResponse>("/videos", { method: "GET", cid });
114      return data.videos ?? [];
115  }
116
117  /** The source-video stream URL for annotation playback. GET /api/source/{video_id}
(P8). The engine
118      * serves a frame-identical proxy; labels stay keyed to the master's MD5
(STUDIO_MEDIA_LIFECYCLE D4). */
119  export function sourceUrl(videoId: string): string {
120      return `${BASE}/source/${encodeURIComponent(videoId)}`;
121  }
122
123  /** Persist finished rally labels as the annotator CSV. POST /api/annotations {video_id,
csv} (P8).
124      * Sent as the byte-compatible CSV (serializeRows) so the engine writes it verbatim
into the corpus. */
125  export async function postAnnotations(videoId: string, csv: string, cid?: string):
Promise<AnnotationsResponse> {
126      return request<AnnotationsResponse>("/annotations", {
127          method: "POST",
128          body: JSON.stringify({ video_id: videoId, csv }),
129          cid,
130      });
131  }
132
133  /** Store a video into the user's chosen medium + register it as a library asset (P4,
D14).
134      * POST /api/assets {uploaded_path? | source_uri?, medium_uri, sport?} ? 201 {video_id,
stored_uri}.
135      * Exactly ONE source: an `uploadedPath` (a just-uploaded local file ? a durable
MD5-keyed copy is
136      * put into the medium, the local upload kept as the working copy) OR a `sourceUri` (a
video already
137      * at rest, registered as-is). Feed `stored_uri` into processVideo(). Fast-fails 400
(bad medium /
138      * both-or-neither source), 403 (hosted posture), 404 (missing file), 507 (no space),
502 (put) ?
139      * surfaced as ApiError. */
140  export async function createAsset(
141      args: { uploadedPath?: string; sourceUri?: string; mediumUri: string; sport?: string
},
142      cid?: string,
143  ): Promise<AssetResponse> {
144      return request<AssetResponse>("/assets", {

```

```

145     method: "POST",
146     body: JSON.stringify({
147         uploaded_path: args.uploadedPath,
148         source_uri: args.sourceUri,
149         medium_uri: args.mediumUri,
150         sport: args.sport,
151     }),
152     cid,
153 });
154 }
155
156 /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
157 export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
158     const data = await request<QueryResponse>("/query", {
159         method: "POST",
160         body: JSON.stringify({ video_id: videoId }),
161         cid,
162     });
163     return data.play_segments ?? [];
164 }
165
166 // Compile-job poll cadence (mirrors useIngestJob): a long ffmpeg stitch must not block
the request.
167 const COMPILE_POLL_MIN_MS = 1000;
168 const COMPILE_POLL_MAX_MS = 5000;
169 const COMPILE_POLL_BACKOFF = 1.5;
170 const COMPILE_POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' compile won't poll
forever
171
172 /** Stitch selected rallies into one reel. ASYNC (#329): POST /api/compile returns 202 +
a job_id;
173 * the ffmpeg stitch runs on the worker and we POLL GET /api/jobs/{id} for the reel ? a
long stitch
174 * would otherwise exceed the ~100 s edge/proxy timeout (524) while the reel actually
built. Resolves
175 * with the job's CompileResult on success; throws ApiError on a failed/timed-out
compile.
176 * `locale` (the UI language) localizes the reel watermark engine-side (omitted when
absent). */
177 export async function compileReel(
178     videoId: string,
179     segmentIds: number[],
180     outputFilename: string,
181     cid?: string,
182     locale?: string,
183 ): Promise<CompileResult> {
184     const c = cid ?? newCorrelationId();
185     const submitted = await request<ProcessResponse>("/compile", {
186         method: "POST",
187         body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename, locale }),
188         cid: c,
189     });
190     return pollCompileJob(submitted.job_id, c);
191 }
192
193 /** Poll a submitted compile job to its terminal state. Returns the reel result on
success; throws an
194 * ApiError for an engine job-failure, a success-shaped FAILURE (done but the stitch
failed / no URL),
195 * or a poll timeout ? so the caller's existing classifyError/errorText mapping
surfaces it like ingest. */
196 async function pollCompileJob(jobId: string, cid: string): Promise<CompileResult> {

```

```

197     const startedAt = Date.now();
198     let delay = COMPILE_POLL_MIN_MS;
199     for (;;) {
200         const job = await getJob(jobId, cid);
201         if (job.status === "done") {
202             const result = (job.result ?? {}) as CompileResult;
203             if (result.status === "failed" || !result.download_url) {
204                 // A real stitch failure carries an engine message ? surface it as an `engine`
detail (status +
205                 // body set) so classifyError shows it verbatim, NOT the no-status "engine
unreachable" fallback.
206                 const msg = result.message || "Highlight stitching failed.";
207                 throw new ApiError(msg, { cid, status: 502, body: msg });
208             }
209             return result;
210         }
211         if (job.status === "failed") {
212             const msg = job.error || "Compile job failed.";
213             throw new ApiError(msg, { cid, status: 502, body: msg });
214         }
215         if (Date.now() - startedAt > COMPILE_POLL_CEILING_MS) {
216             throw new ApiError("Compile timed out.", { cid, status: 504, body: "Compile timed
out." });
217         }
218         await new Promise((resolve) => setTimeout(resolve, delay));
219         delay = Math.min(delay * COMPILE_POLL_BACKOFF, COMPILE_POLL_MAX_MS);
220     }
221 }
222
223 /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
224 export function highlightUrl(videoId: string, filename: string): string {
225     return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
226 }
227
228 /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
229 export async function getConfig(cid?: string): Promise<EngineConfig> {
230     return request<EngineConfig>("/config", { method: "GET", cid });
231 }
232
233 /** What THIS box can do ? drives the first-run wizard. GET /api/capabilities
(PACKAGING_PLAN P2c). */
234 export async function getCapabilities(cid?: string): Promise<EngineCapabilities> {
235     return request<EngineCapabilities>("/capabilities", { method: "GET", cid });
236 }
237
238 /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
239 export function minShotCount(cfg: EngineConfig): number | undefined {
240     return cfg.stitching?.min_shot_count;
241 }
242
243 // ?? chunked upload (backend/api/uploads.py) ?????????????????????????????????????????
244 // A browser uploads a LOCAL file in sequential parts ? max_part_mb (free-tier edge
proxies cap
245 // bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into
/api/process.
246
247 /** Start a chunked upload. 400 = bad extension; 413 = over the size limit
[uploads.py:64]. */
248 export async function uploadInit(filename: string, totalSizeBytes: number, cid?:
string): Promise<UploadInitResponse> {
249     return request<UploadInitResponse>("/upload/init", {
250         method: "POST",

```

```

251     body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
252     cid,
253   });
254 }
255
256 /** Append ONE part (raw bytes ? not JSON). Parts are strictly sequential 0,1,2,?
[uploads.py:89]. */
257 export async function uploadPart(
258   uploadId: string,
259   index: number,
260   chunk: Blob,
261   cid?: string,
262   signal?: AbortSignal,
263 ): Promise<UploadPartResponse> {
264   const c = cid ?? newCorrelationId();
265   const url = `${BASE}/upload/${encodeURIComponent(uploadId)}/part/${index}`;
266   log("info", "api.request", { cid: c, method: "PUT", url });
267   let res: Response;
268   try {
269     res = await fetch(url, {
270       method: "PUT",
271       body: chunk,
272       headers: { "content-type": "application/octet-stream", "x-correlation-id": c },
273       signal,
274     });
275   } catch (err) {
276     log("error", "api.network_error", { cid: c, method: "PUT", url, error: String(err)
});
277     throw new ApiError(`Network error calling PUT ${url}`, { cid: c, cause: err });
278   }
279   if (!res.ok) {
280     const body = await res.text().catch(() => "");
281     log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status,
body: body.slice(0, 500) });
282     throw new ApiError(`PUT ${url} failed: ${res.status}`, { cid: c, status: res.status,
body });
283   }
284   log("info", "api.response", { cid: c, method: "PUT", url, status: res.status });
285   return (await res.json()) as UploadPartResponse;
286 }
287
288 /** Finalize: returns {path, video_id} ? feed `path` to processVideo() [uploads.py:123].
*/
289 export async function uploadComplete(uploadId: string, totalParts: number, cid?:
string): Promise<UploadCompleteResponse> {
290   return
request<UploadCompleteResponse>(`/upload/${encodeURIComponent(uploadId)}/complete`, {
291     method: "POST",
292     body: JSON.stringify({ total_parts: totalParts }),
293     cid,
294   });
295 }
296
297 /** Abort an in-flight upload (best-effort cleanup of the server-side partial)
[uploads.py:147]. */
298 export async function uploadAbort(uploadId: string, cid?: string): Promise<void> {
299   await request<unknown>(`/upload/${encodeURIComponent(uploadId)}`, { method: "DELETE",
cid });
300 }
301
302 export interface UploadProgress {
303   sentBytes: number;
304   totalBytes: number;
305 }
306

```



```

307     /** Upload a local File end to end: init ? sequential parts (byte progress, abortable) ?
complete.
308     * On ANY failure or abort it best-effort DELETes the server-side partial, then
rethrows so the
309     * caller can classify it (the engine's 400/413 detail survives). Returns {path,
video_id}.
310     * Note: a per-part 413 (over the per-part limit) aborts the upload server-side ?
recovery is a
311     * fresh init (re-pick the file), which the UI's retry does. */
312     export async function uploadFile(
313         file: File,
314         opts: { cid?: string; signal?: AbortSignal; onProgress?: (p: UploadProgress) => void }
= {}),
315     ): Promise<UploadCompleteResponse> {
316         const cid = opts.cid ?? newCorrelationId();
317         const init = await uploadInit(file.name, file.size, cid);
318         // Honor the engine's max_part_mb, but never let a missing/NaN value collapse the math
to NaN (which
319         // would silently upload zero parts) ? fall back to the engine's 95 MB default.
320         const partMb = Number.isFinite(init.max_part_mb) && init.max_part_mb > 0 ?
init.max_part_mb : 95;
321         const partBytes = partMb * 1024 * 1024;
322         const totalParts = Math.max(1, Math.ceil(file.size / partBytes));
323         log("info", "upload.start", { cid, filename: file.name, size: file.size, parts:
totalParts, part_mb: init.max_part_mb });
324         let sent = 0;
325         try {
326             for (let i = 0; i < totalParts; i++) {
327                 if (opts.signal?.aborted) throw new DOMException("Upload aborted", "AbortError");
328                 const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
329                 await uploadPart(init.upload_id, i, chunk, cid, opts.signal);
330                 sent += chunk.size;
331                 opts.onProgress?.({ sentBytes: sent, totalBytes: file.size });
332             }
333             const done = await uploadComplete(init.upload_id, totalParts, cid);
334             log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size:
done.size_bytes });
335             return done;
336         } catch (err) {
337             log("warn", "upload.aborting_partial", { cid, upload_id: init.upload_id, error:
String(err) });
338             await uploadAbort(init.upload_id, cid).catch(() => {});
339             throw err;
340         }
341     }
342

```

6. user (2026-07-02T03:02:35.769Z)

```

1     // The ONE place a classified error (lib/errors.ts) becomes user-facing text. Both
surfaces use it:
2     // the ingest flow (IngestPanel) and the build flow (App.onBuild ? CompileResult), so a
5xx / engine-
3     // down failure reads the same friendly way everywhere instead of a raw "HTTP 500" /
fetch message.
4     //
5     // `engine` carries the engine's OWN message (the most specific) ? shown verbatim, never
translated.
6     // Every other code maps to a friendly localized line in the shared `errors.*`
namespace. Kept out of
7     // errors.ts so that module stays i18n-free and unit-testable without a translation
context.
8     import type { TFunction } from "i18next";
9     import type { IngestError } from "../errors";
10

```

```

11     export function errorText(t: TFunction, e: IngestError): string {
12         if (e.code === "engine") return e.detail ?? t("errors.engineGeneric");
13         return t(`errors.${e.code}`);
14     }
15

```

7. user (2026-07-02T03:02:38.602Z)

501c014 feat(web): A5 ? friendly engine-error mapping across build + load (shared errors.*) (#62)
4525c5d feat(web): B-P2 ? in-UI bucket ingest (point at a gs:// video ? process ? load) (#56)
f9eea14 web: resilient video-summary readers + show fps in the library (#140)

master...origin/master

8. user (2026-07-02T03:02:55.847Z)

```

{"body":"***From the first real hosted reel build.** On a Cloudflare **524 (origin timeout)**
during \"Build reel\", the SPA surfaced the **raw Cloudflare error-page HTML** (\"Build failed\"
with a wall of `<!DOCTYPE html>?524?`). Ugly and alarming ? *especially* because the reel
actually built (the 524 only timed out the HTTP response).\n\nFix: the SPA error mapping
(`lib/errors.ts` / `errorText.ts`, the A5 layer) should detect a **non-JSON / 5xx / 524 /
timeout** response and show a **friendly message** instead of rendering the body ? e.g. *\"This
is taking longer than usual ? your reel may still be building. We'll have async builds soon; for
now, try again in a moment.\"* Bonus: on a compile timeout, **probe for the finished reel**
(`/api/highlights/{video_id}/{name}`) before declaring failure.\n\nRelates to engine #329 (async
`/api/compile` removes the 524 at the source); this is the client-side defense regardless. Same
`loud-but-friendly failures` bar as A5.", "labels": [], "state": "OPEN", "title": "SPA: show a
friendly failure notice on a timeout/5xx (Cloudflare 524), not the raw Cloudflare error
HTML", "updatedAt": "2026-06-22T14:14:16Z"}

```

9. user (2026-07-02T03:02:55.977Z)

```

khelsutra\web\src\App.tsx:116:         const e = classifyError(err);
khelsutra\web\src\App.tsx:117:         setLoadState({ status: "error", error: errorText(t, e) });
khelsutra\web\src\App.tsx:207:        const e = classifyError(err);
khelsutra\web\src\App.tsx:208:        setBuild({ status: "error", error: errorText(t, e) });
khelsutra\web\src\lib\errors.test.ts:20:    expect(classifyError(new ApiError("net", { cid: "c"
}))).toEqual({ code: "networkError" });
khelsutra\web\src\lib\errors.test.ts:24:    expect(classifyError(err)).toEqual({ code: "engine",
detail: "unsupported storage URI scheme" });
khelsutra\web\src\lib\errors.test.ts:27:    expect(classifyError(new ApiError("x", { status: 500
}))).toEqual({ code: "networkError" });
khelsutra\web\src\lib\errors.test.ts:28:    expect(classifyError(new ApiError("x", { status: 502
}))).toEqual({ code: "networkError" });
khelsutra\web\src\lib\errors.test.ts:32:    expect(classifyError(err)).toEqual({ code: "engine",
detail: "failed to persist job record" });
khelsutra\web\src\lib\errors.test.ts:35:    expect(classifyError(new ApiError("x", { status: 404
}))).toEqual({ code: "engine", detail: "HTTP 404" });
khelsutra\web\src\lib\errors.test.ts:38:    expect(classifyError(new Error("boom"))).toEqual({
code: "engine", detail: "Error: boom" });
khelsutra\web\src\components\IngestPanel.tsx:90:        setStoreError(classifyError(err));
khelsutra\web\src\components\IngestPanel.tsx:138:    const errorMsg = errorState ? errorText(t,
errorState) : null;
khelsutra\web\src\lib\errors.ts:30:export function classifyError(err: unknown): IngestError {
khelsutra\web\src\lib\errorText.test.ts:18:    expect(errorText(i18n.t, { code: "engine",
detail: "min_shot_count too low" })).toBe("min_shot_count too low");
khelsutra\web\src\lib\errorText.test.ts:22:    const s = errorText(i18n.t, { code: "engine" });
khelsutra\web\src\lib\errorText.test.ts:29:    expect(errorText(i18n.t, { code: "networkError"
})).toMatch(/is it running/i);
khelsutra\web\src\lib\errorText.test.ts:34:    const s = errorText(i18n.t, { code: });
khelsutra\web\src\lib\errorText.test.ts:42:    expect(errorText(i18n.t, { code: "networkError"
})).toBe("KhelSutra ??????????????. ??? ?????????????????");
khelsutra\web\src\lib\errorText.ts:11:export function errorText(t: TFunction, e: IngestError):
string {

```

```

khelsutra\web\src\hooks\useIngestJob.ts:83:          fail(classifyError(err),
"ingest.poll_error");
khelsutra\web\src\hooks\useIngestJob.ts:122:         fail(classifyError(err),
"ingest.submit_error");
khelsutra\web\src\hooks\useUpload.ts:67:         setPhase({ status: "error", error:
classifyError(err) });

```

10. assistant (2026-07-02T03:03:30.276Z)

Let me read the errors test to check the 5xx-with-body expectations, which the proposed fix would touch.

11. user (2026-07-02T03:03:30.951Z)

```

1      import { describe, it, expect } from "vitest";
2      import { parseDetail, classifyError } from "../errors";
3      import { ApiError } from "../api/client";
4
5      describe("parseDetail", () => {
6          it("extracts FastAPI's {detail}", () => {
7              expect(parseDetail(JSON.stringify({ detail: "unsupported storage URI scheme"
8          }))).toBe(
9              "unsupported storage URI scheme",
10             );
11         });
12         it("returns the raw body when it isn't JSON or has no string detail", () => {
13             expect(parseDetail("plain text")).toBe("plain text");
14             expect(parseDetail(JSON.stringify({ other: 1 }))).toBe(JSON.stringify({ other: 1
15         })));
16         expect(parseDetail(JSON.stringify({ detail: { nested: true }
17     }))).toBe(JSON.stringify({ detail: { nested: true } }));
18     });
19     });
20     });
21     });
22     describe("classifyError", () => {
23         it("a network throw (ApiError with no status) ? networkError", () => {
24             expect(classifyError(new ApiError("net", { cid: "c" }))).toEqual({ code:
25 "networkError" });
26         });
27         it("an HTTP error with a body ? engine + the FastAPI detail", () => {
28             const err = new ApiError("x", { status: 400, body: JSON.stringify({ detail:
29 "unsupported storage URI scheme" }) });
30             expect(classifyError(err)).toEqual({ code: "engine", detail: "unsupported storage
31 URI scheme" });
32         });
33         it("a bare 5xx with no body ? networkError (friendly 'is the engine running?', not
34 'HTTP 500')", () => {
35             expect(classifyError(new ApiError("x", { status: 500 }))).toEqual({ code:
36 "networkError" });
37             expect(classifyError(new ApiError("x", { status: 502 }))).toEqual({ code:
38 "networkError" });
39         });
40         it("a 5xx WITH a detail still surfaces the engine's message", () => {
41             const err = new ApiError("x", { status: 500, body: JSON.stringify({ detail: "failed
42 to persist job record" }) });
43             expect(classifyError(err)).toEqual({ code: "engine", detail: "failed to persist job
44 record" });
45         });
46         it("a 4xx with no body keeps the status (a client-side mistake, not unreachable)", ()
47 => {
48             expect(classifyError(new ApiError("x", { status: 404 }))).toEqual({ code: "engine",
49 detail: "HTTP 404" });
50         });
51         it("a non-ApiError ? engine + the stringified error", () => {
52             expect(classifyError(new Error("boom"))).toEqual({ code: "engine", detail: "Error:

```

```
boom" });  
39      });  
40      });  
41
```

12. user (2026-07-02T03:04:04.298Z)

Structured output provided successfully